

pattern-drift

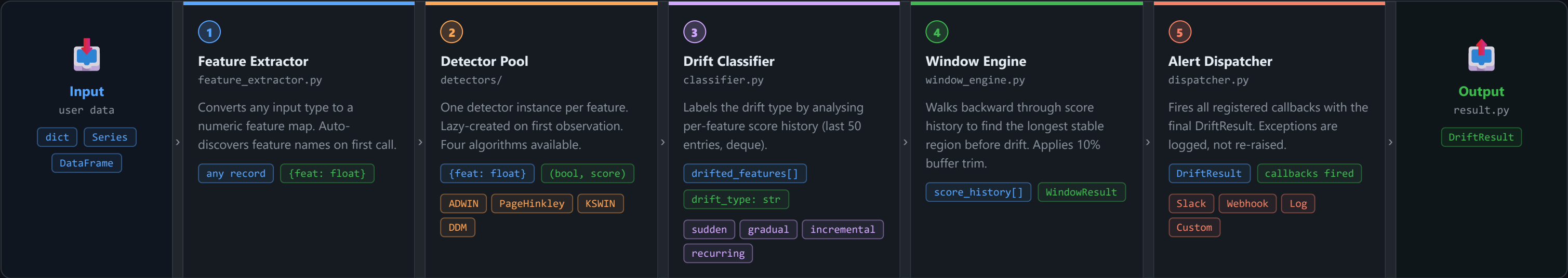
Architecture & Data-Flow Reference

v0.1.0

Pure stdlib · no mandatory deps

Python ≥ 3.9

5-STAGE DETECTION PIPELINE



CONSTRUCTOR PARAMETERS

method	"ADWIN" "PageHinkley" "KSWIN" "DDM"
sensitivity	0.002 – detection threshold
min_window	30 – warmup before reporting
max_window	10 000 – history cap
features	Optional[List[str]] – None = auto
callbacks	Optional[List[Callable]]

INTERNAL STATE

_extractor	FeatureExtractor
_detectors	Dict[str, BaseDetector]
_classifier	DriftClassifier
_window_engine	RetrainingWindowEngine
_dispatcher	AlertDispatcher
_score_history	List[Dict[str, float]]
_n_updates	int

PUBLIC API

PUB	update(data) → DriftResult
PUB	reset() → None
PUB	plot_drift_timeline() → None requires matplotlib
PUB	export_report(path: str) → None .json .csv
PUB	set_reference(data) → None KSWIN only
CLS	from_config(path: str) → DriftMonitor requires pyyaml
PRV	_update_single(record) → DriftResult
PRV	_get_or_create_detector(feature) → BaseDetector
PRV	_export_json(path)
PRV	_export_csv(path)

_UPDATE_SINGLE() FLOW

```
_update_single(record): features = _extractor.extract(record) # stage 1
for feat, val in features.items(): det = _get_or_create_detector(feat) # stage 2
drift, score = det.update(val) classifier.record(feat, drift, score) # stage 3
if drift and n_updates ≥ min_window: drifted_features.append(feat)
score_history.append(step_scores)
if not drifted_features: return no-drift result
drift_type = classifier.classify(drifted_features)
window = window_engine.find_window(...) # stage 4
result = DriftResult(...) dispatcher.dispatch(result) # stage 5
return result
```

BASEDETECTOR (ABSTRACT, DETECTORS/BASE.PY)

ABS

update(value: float) → Tuple[bool, float]

ABS

reset() → None

Returns (drift_detected, drift_score)
All four concrete detectors inherit this interface.

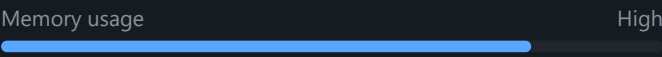


ADWIN

Adaptive Windowing

Maintains a sliding window. For every split point, computes left/right means and compares to a Hoeffding-bound threshold ϵ . Drift = $|m_0 - m_1| \geq \epsilon$. Discards left portion on drift.

Params	delta=0.002, max_window=10k
Memory	$O(\text{max_window})$
Speed	$O(n)$ per update
Detects	Gradual mean shift
Warmup	Gradual
State	deque[float], total: float

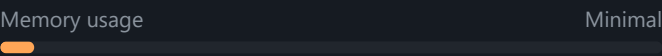


PageHinkley

Cumulative Sum Test

Tracks cumulative deviation from running mean minus δ . Drift = $U > \lambda$, where $U = \text{cumsum} - \text{min_cumsum}$. Exponentially weighted mean via α forgetting factor.

Params	delta=0.005, lambda_=50, alpha=1.0
Memory	$O(1) - 3$ scalars
Speed	$O(1)$ per update
Detects	Sudden mean shift
Warmup	Immediate
State	n, mean, cumsum, min_cumsum

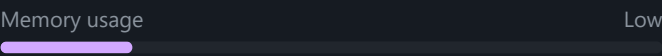


KSWIN

KS-Windowed Test

Runs a two-sample KS test between the most recent stat_size values and a reference window. p-value $< \alpha \rightarrow$ drift. stdlib KS implementation (no scipy).

Params	alpha=0.005, window=100, stat=30
Memory	$O(\text{window_size})$
Speed	$O(n \cdot \log n)$ per update
Detects	Any distribution change
Warmup	window_size observations
State	deque[float], reference list

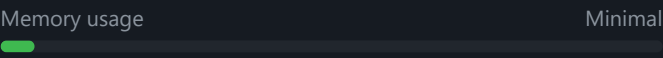


DDM

Drift Detection Method

Monitors binary error rate. Drift = $p_t + \sigma_t > p_{\text{min}} + \text{drift_level} \times \sigma_{\text{min}}$. Tracks minimum baseline ($p_{\text{min}}, \sigma_{\text{min}}$) since last reset.

Params	warn=2.0, drift=3.0, min=30
Memory	$O(1) - 5$ scalars
Speed	$O(1)$ per update
Detects	Classifier error rate $\uparrow$
Warmup	min_num_instances (30)
State	n, p, σ , p_min, σ_{min}



DRIFTCLASSIFIER

Maintains per-feature score history (deque, maxlen=50). Priority: sudden > recurring > incremental > gradual.

- PUB

record(feature, drift_detected, score) → None
- PUB

classify(drifted_features) → Optional[str]
- PUB

reset() → None
- PRV

_classify_feature(feature) → str



Sudden

Last 2 scores jump: prev < 0.2 → curr > 0.6. Abrupt distribution change.



Recurring

Feature has drifted more than once total (_drift_count > 1). Cyclic or seasonal concept drift.



Incremental

Last 5 scores are monotonically increasing. Slow continuous drift over time.



Gradual

Default fallback. Slow change not fitting the above patterns.

RETRAININGWINDOWENGINE

Finds optimal stable data window for model retraining after drift is confirmed.

- PUB

find_window(score_history, drift_index) → Optional[RetrainingWindowResult]

Algorithm

- 1

Walk backward from `drift_index`
- 2

Stable = all features' max score < sensitivity
- 3

Stop at first unstable observation
- 4

Trim `buffer_pct` × `n` (10%) from each end
- 5

Return None if < 2 stable samples remain
- 6

Confidence = fraction of truly stable samples in raw region

Constructor Parameters

- sensitivity

0.002 – stable threshold per feature
- buffer_pct

0.10 – 10% trim from each end

ALERTDISPATCHER

- PUB

register(callback: Callable) → None
- PUB

dispatch(result: DriftResult) → None
- PUB

reset() → None

Built-in Callback Factories

- 

slack_callback(webhook_url)
POSTs a formatted :warning: message with drift type, features, score and timestamp to a Slack incoming webhook.
requires: requests
- 

webhook_callback(url)
POSTs full JSON DriftResult payload (including retraining window bounds) to any HTTP endpoint.
requires: requests
- 

log_callback(level="warning")
Logs via Python's logging module. Level configurable: debug / info / warning / error.
stdlib only
- 

Custom callbacks
Any `Callable[[DriftResult], None]` is valid. Exceptions are caught and logged — never crash the monitor.

DRIFTRESULT

FIELD	TYPE	DESCRIPTION
drift_detected	bool	True when any feature drifts past min_window
drift_type	Optional[str]	"sudden" "gradual" "incremental" "recurring"
drifted_features	List[str]	Names of features that triggered detection
drift_score	float	Max normalised score across all drifting features
retraining_window	Optional[...]	Suggested window result, or None if insufficient data
timestamp	datetime	UTC datetime of detection

RETRAININGWINDOWRESULT

FIELD	TYPE	DESCRIPTION
start	int	Start index into monitor's score_history
end	int	End index (exclusive) into score_history
n_samples	int	Number of stable samples in the window
confidence	float	0.0–1.0 stability fraction (1.0 = perfectly clean)

FEATUREEXTRACTOR

Handles any input type, filters to numeric columns only, and manages auto-discovery of feature names.

PUB	extract(record) → Dict[str, float]
PROP	features → Optional[List[str]]
PUB	reset() → None
PRV	_to_dict(record) → dict (static)
PRV	_filter_numeric(d: dict) → dict (static)

Input handling



DataFrame
Processed row-by-row
returns first/last result



Series
Converted to dict
then filtered



dict
Filtered to
numeric values

NaN and non-numeric values are silently excluded. Auto-discovered features are sorted and cached; explicit features persist through `reset()`.

DRIFTDETECTOR PASS-THROUGH TRANSFORMER

Wraps DriftMonitor as a sklearn-compatible `TransformerMixin`. Data passes through unchanged — drift events are handled purely via callbacks.

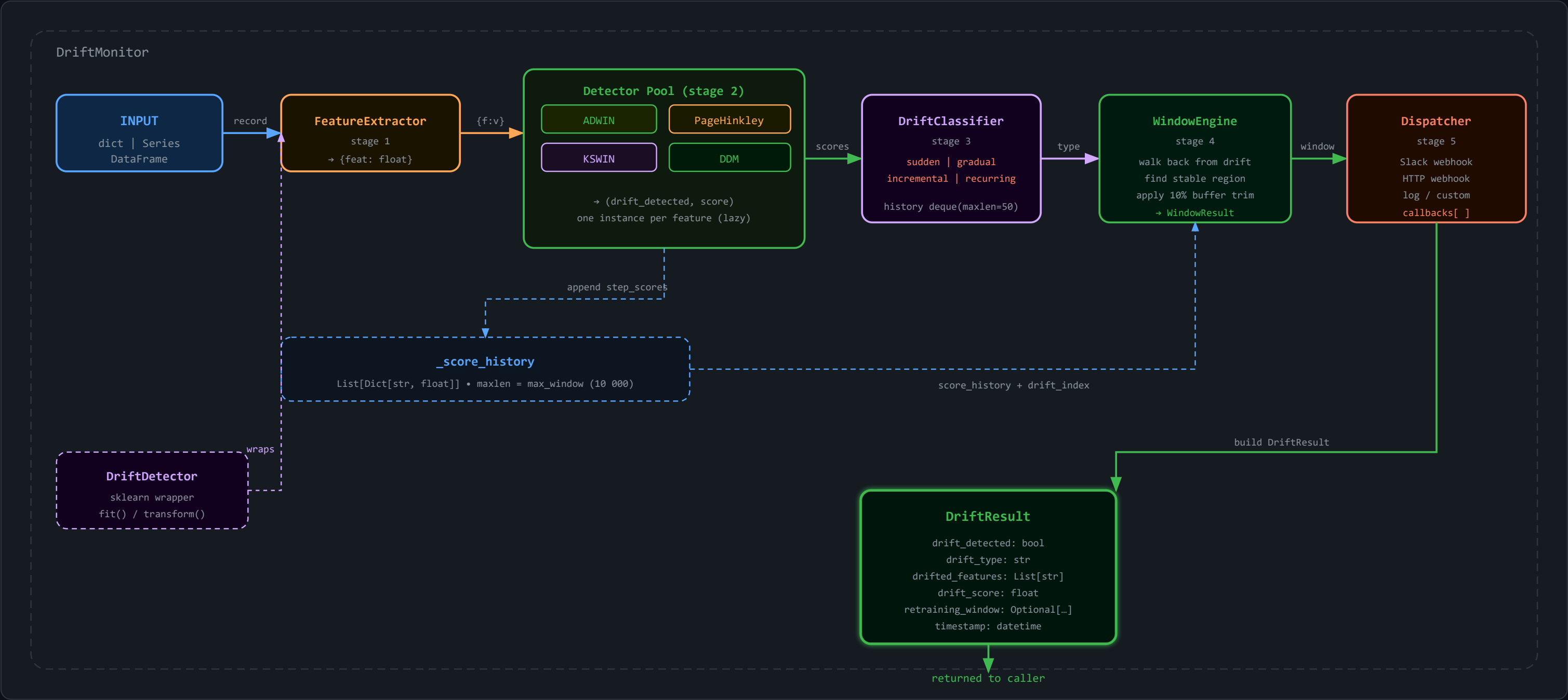
PUB	fit(X, y=None) → self	initialise monitor, set reference
PUB	transform(X, y=None) → X	run monitor, return X unchanged
PUB	get_params(deep=True) → dict	
PUB	set_params(**params) → self	
PROP	monitor → DriftMonitor	

Usage in sklearn Pipeline

```
from pattern_drift import DriftDetector from sklearn.pipeline import Pipeline pipe = Pipeline([
("drift", DriftDetector( method="ADWIN", callbacks=[log_callback()] )), ("model",
your_estimator), ]) pipe.fit(X_train) pipe.transform(X_stream) # drift detected silently
```

requires: scikit-learn ≥ 1.1 (optional extra: pip install pattern-drift[sklearn])

FULL DATA & CONTROL FLOW



DEPENDENCY MAP & OPTIONAL EXTRAS

CORE (ZERO MANDATORY DEPS)

The entire detection pipeline runs on Python stdlib only. All optional deps add extra entry points.

- collections.deque
- math
- logging
- datetime
- abc
- dataclasses
- typing
- json
- csv

OPTIONAL EXTRAS

pandas ≥1.5	DataFrame & Series input support
scikit-learn ≥1.1	DriftDetector sklearn transformer
matplotlib ≥3.5	plot_drift_timeline() visualisation
requests ≥2.28	Slack & webhook alert callbacks
pyyaml ≥6.0	DriftMonitor.from_config(yaml)
all	All of the above

DETECTOR COMPARISON MATRIX

Aspect	ADWIN	PageHinkley	KSWIN	DDM
Detects	Gradual mean shift	Sudden mean shift	Any distribution Δ	Error rate 1
Memory	$O(\text{max_window})$	$O(1)$	$O(\text{window_size})$	$O(1)$
Speed	$O(n)$ per update	$O(1)$ per update	$O(n \cdot \log n)$ per update	$O(1)$ per update
Input type	Feature values	Feature values	Feature values	Binary (0/1 error)
Window-based	✓ yes	✗ no	✓ yes	✗ no
Warmup	Gradual	Immediate	window_size obs.	min_num_instances
Sensitivity param	$\text{delta}=0.002$	$\text{delta}=0.005, \lambda=50$	$\text{alpha}=0.005$	$\text{drift_level}=3.0$
Best use case	Continuous monitoring	Trend / step changes	Shape / variance Δ	ML model degradation

● Input / core pipeline ● Stage 2 detectors ● Classification / sklearn ● Output / window ● Alerts / dispatcher ● Optional dependency

pattern-drift v0.1.0 · Pure Python stdlib · 5-stage pipeline · 4 detectors · 34 tests passing